```
initWithTitle:@"Hello"
message:@"This is an alert view"
delegate:self
cancelButtonTitle:@"OK"
otherButtonTitles:@"No", @"Cancel", nil];
```

To determine which button was tapped by the user, you need to handle the relevant method(s) that will be fired by the alert view when the buttons are clicked. The UIAlertViewDelegate protocol handles these methods. They are:

- alertView:clickedButtonAtIndex:
- willPresentAlertView:
- didPresentAlertView:
- alertView:willDismissWithButtonIndex:
- alertView:didDismissWithButtonIndex:
- alertViewCancel:

To implement any of these methods in the UIAlertViewDelegate protocol, you need to ensure that your class, in this case the View Controller, conforms to this protocol. This is made possible by using angle brackets (<>) as follows:

```
@interface UsingViewsViewController : UIViewController
< UIAlertViewDelegate > { //this class conforms to the
UIAlertViewDelegate protocol
}
```

You can even conform to more than one delegate by separating the protocols with commas, like <UIAlertViewDelegate, UITableViewDataSource>.

Once the class conforms to a protocol, you can implement the method in your class without any problem:

```
- (void)alertView:(UIAlertView *)alertView
clickedButtonAtIndex:(NSInteger)buttonIndex {
NSLog([NSString stringWithFormat:@"%d", buttonIndex]);
}
```

Delegate

A delegate is just another object that has been assigned by an object as the object responsible for handling events. In the UIAlertView example that we discussed before:

```
UIAlertView *alert = [[UIAlertView alloc]
initWithTitle:@"Hello"
message:@"This is an alert view"
```